

Contents

guide SGLang: Efficient Execution of Structured Language Model Programs	1
0. 先给你一个抓手	1
1. 当时的语境: 从聊天到 LM program	2
2. 论文地图	2
3. 前端语言到底暴露了什么	3
4. Interpreter 和 compiler	3
5. RadixAttention: 为什么 KV cache 可以复用	4
6. Figure 3 用文字重建	5
7. LRU eviction 和 refcount	6
8. Cache-aware scheduling	6
9. Frontend hint 为什么重要	6
10. Compressed FSM 和 jump-forward decoding	7
11. API speculative execution	7
12. 实验证据链	8
12.1 Setup	8
12.2 End-to-end results	9
12.3 Larger and multi-modal models	9
12.4 Production deployment	9
12.5 Ablations	9
13. 和本仓库代码的连接	10
14. 这篇论文没有证明什么	11
15. 和后续工作的关系	12
16. 用 AI agent 学这篇论文的正确方式	12
17. 读完后的闭卷复述模板	13

guide_SGLang: Efficient Execution of Structured Language Model Programs

原论文: SGLang: Efficient Execution of Structured Language Model Programs
作者: Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, Ying Sheng
版本: arXiv v2, 2024-06-06
本地原文 PDF: [learning/sglang-radixattention/paper/01_sglang.pdf](#)
本地导读 PDF: [learning/sglang-radixattention/paper/guide_01_sglang.pdf](#)
本地核心代码: [learning/sglang-radixattention/src/radix_tree.py](#)
本地机制实验: [learning/sglang-radixattention/src/sglang_original_minimal.py](#)

0. 先给你一个抓手

SGLang 这篇论文的核心不是“又写了一个 prompt 框架”。它真正关心的是:

LLM 应用已经变成程序了。

程序里有多次模型调用、分支、并行、工具、结构化输出。

如果 runtime 不理解这些结构, 就会重复算大量 prefix 和 KV cache。

SGLang 的回答是前端语言和后端 runtime 联合设计:

frontend:

用 gen/select/fork/join/image/video 表达 LM program

runtime:

用 RadixAttention 复用 KV cache

用 compressed FSM 加速结构化输出

用 API speculative execution 减少黑盒 API 多调用成本

如果只能记一句话，记这个：

SGLang exposes program structure so the runtime can see cache reuse and scheduling opportunities that a plain OpenAI-style API hides.

1. 当时的语境：从聊天到 LM program

早期 LLM 使用方式很像单轮聊天：

prompt -> one generation

但真实应用很快变复杂：

- few-shot benchmark: 很多请求共享同一组 examples。
- ReAct agent: system prompt、工具说明、历史观察不断被追加。
- tree-of-thought: 同一个 prefix fork 出多个推理分支。
- self-consistency: 同一个问题采样多条答案。
- structured output: JSON、schema、正则约束。
- RAG pipeline: 检索上下文、模板、多个生成调用组合。
- multi-modal: 同一张图像或视频被问多个问题。

这些都不是单次 completion，而是 Language Model Programs, 简称 LM programs。论文总结 LM programs 有两个共性：

- 多个 LLM call 交织着控制流。
- 输入和输出通常有结构，方便和软件系统组合。

传统 serving engine 如 vLLM、TGI、TensorRT-LLM 很强，但它们通常只看到一个个请求。它们不知道这些请求来自同一个程序，不知道哪里 fork 了，不知道多个调用共享了 system prompt 或 few-shot examples。结果就是很多 prefix 被重复 prefill，KV cache 用完就丢。

SGLang 的切入点是：如果前端语言把程序结构显式表达出来，后端 runtime 就能自动复用和调度。

2. 论文地图

按这个顺序读：

- Section 1: 介绍 LM programs 的兴起，以及当前表达和执行都低效。
- Section 2: SGLang programming model, 讲 gen、select、fork、join、image、video。
- Section 3: RadixAttention, 讲 KV cache 复用、radix tree、LRU eviction、cache-aware scheduling。
- Theorem 3.1: 离线 batch 中 DFS order 可以达到最优 cache hit rate; longest-shared-prefix-first 等价于 DFS。
- Section 4: compressed FSM, 加速 JSON/regex constrained decoding。
- Section 5: API speculative execution, 面向 GPT-4/GPT-3.5 这种黑盒 API。
- Section 6: evaluation, 覆盖 Llama、Mixtral、LLaVA、GPT-3.5 和多类 workload。
- Section 7: related work, 和 vLLM、PromptCache、HydraGen、Guidance、LMQL 等比较。

- Section 8: future directions.
- Appendix A: KV cache 背景、RadixAttention 证明、distributed RadixAttention.
- Appendix B: compressed FSM 细节.
- Appendix D: compiler mode 和 IR.

图表抓重点:

- Figure 1: 前端语言加后端 runtime 的系统架构。
- Figure 2: multi-dimensional essay judge, 展示 select、fork、gen(regex) 和三个 runtime 优化机会。
- Table 1: LMQL、Guidance、SGLang 的 primitives 和 runtime backend 对比。
- Figure 3: RadixAttention radix tree 的九步演化, 是理解论文的关键图。
- Figure 4: normal FSM 和 compressed FSM 的差别。
- Figure 5/6/7/12: throughput 和 latency 对比。
- Figure 8: cache hit rate 与 latency/throughput, 以及 RadixAttention ablation.
- Figure 13: achieved cache hit rate 和 optimal cache hit rate.
- Figure 14: SGLang program 到 IR graph.

3. 前端语言到底暴露了什么

SGLang 是嵌入 Python 的 DSL。它不是要替代 Python, 而是给 LM program 常用动作提供 primitives.

核心 primitives:

- extend 或 +=: 追加字符串、图像、视频或 primitive 到 prompt state.
- gen(name, ...): 调模型生成, 并把结果存到变量 name.
- select(name, choices=...): 在候选项里选概率最高的选项。
- fork(k): 把当前 prompt state 复制成多个分支。
- join: 合并分支。
- image 和 video: 支持多模态输入。
- regex: 对 gen 的输出施加正则约束。

Figure 2 的 essay judge 很适合作为入口。它做了这些事:

1. 给模型图像和 essay
2. select 判断 essay 是否相关
3. 如果相关, fork 成三个维度并行评审
4. merge 三个 judgment
5. gen summary 和 grade
6. 用 regex 约束最终 JSON 输出

这段程序对 runtime 暴露了三个优化机会:

- fork 后多个分支共享同一个 prefix.
- JSON 输出里有很多固定 literal, 可以 compressed FSM 加速。
- API 模型可以在前一次调用里多生成一段, 供后续 primitive 复用。

普通字符串拼接也能写出同样逻辑, 但 runtime 看不见结构。SGLang 的前端价值, 就是把这些结构变成 runtime 可用的信号。

4. Interpreter 和 compiler

论文默认使用 interpreter mode.

直觉:

```
prompt state is an asynchronous stream
extend/gen/select are submitted to the stream
Python code can continue before generation completes
fetching a variable blocks when needed
```

它有点像 CUDA kernel launch: 调用先提交, 真正需要结果时再同步。这样 fork 出来的多个分支可以并行执行。

SGLang 也支持 compiler mode。程序可以被 tracing 成 computational graph, 再由 graph executor 执行。Appendix D 讲了 IR:

- 常量文本、参数、Gen、Select、变量访问都是 graph node。
- fork 会创建多个 stream。
- join 或变量读取形成同步边。

Compiler mode 的好处包括:

- 减少 Python interpreter 开销。
- 支持 graph rewriting。
- 支持程序序列化。
- 理论上可以做 scheduling 和 memory planning。

论文还提到一个有趣但谨慎的方向: code movement for prefix sharing。比如把 {question} 放在更后面, 让更多 instruction 变成可共享 prefix。这种优化不严格保持原程序语义, 因为自然语言提示顺序可能影响模型行为, 所以作者用 GPT-4 辅助重排, 并把它放在附录探索。

5. RadixAttention: 为什么 KV cache 可以复用

Transformer 自回归推理里, 每个 prefix token 会产生 key/value 中间张量。后续 token 解码时会复用这些张量, 这就是 KV cache。

关键性质:

KV cache for a token depends only on all previous tokens.

所以如果两个请求有相同 prefix:

```
request A:
    system + examples + question A
```

```
request B:
    system + examples + question B
```

那么 system + examples 的 KV cache 可以复用。普通 serving engine 常常在请求结束后丢掉 KV cache, 或者只支持简单的一层 prefix cache。SGLang 要处理的是更复杂的共享模式:

- 多轮聊天共享历史前缀。
- few-shot benchmark 共享 examples。
- fork 分支共享 fork 之前的 prompt。
- self-consistency 共享问题和部分生成。
- multi-modal 请求共享同一图像 token。

RadixAttention 的做法:

```
store token sequences in a radix tree
each edge is a token segment, not just one token
edge/node points to corresponding KV cache pages
```

```
match incoming prompt against tree
reuse matched KV
compute only unmatched suffix
insert new suffix after request finishes
```

Radix tree 比普通 trie 更紧凑，因为边可以存一段 token。

6. Figure 3 用文字重建

Figure 3 展示了九个时间点。你可以这样读：

```
step 1:
    tree empty

step 2:
    first chat turn enters tree
    system + user hello + assistant hi becomes one edge

step 3:
    new prompt shares the first turn
    runtime reuses that KV and appends new turn

step 4:
    another chat session shares only system prompt
    old edge is split so both sessions share system part

step 5:
    memory limit forces LRU leaf eviction
    common ancestors remain reusable

step 6:
    few-shot query arrives
    it may share little with chat branch

step 7:
    multiple few-shot queries share examples
    node is split to expose common examples

step 8:
    first chat session continues
    stale second-session leaves get evicted

step 9:
    self-consistency samples more answers
    question prefix is reused across sampled branches
```

最重要的是 split。没有 split，两个请求如果只共享一个 edge 的前半段，就没法把共享前缀单独拿出来。RadixAttention 通过 split 把共享部分变成一个节点，后面的不同 suffix 变成分支。

本仓库的 radix_tree.py 对应这个逻辑：

- match(prefix): 找最长匹配。

- `_split(node, at)`: 在 partial match 处拆边。
- `insert(prefix)`: 插入新请求并记录命中 token 数。
- `acquire/release`: refcount 保护正在使用的节点。
- `evict(want_tokens)`: LRU leaf-first eviction。
- `hit_rate`: $\text{cached prompt tokens} / \text{total prompt tokens}$ 。

7. LRU eviction 和 refcount

GPU memory 很快会被 KV cache 填满。SGLang 没有预先划一个固定 cache pool, 而是让 cached tokens 和 running requests 共享同一个 memory pool。

当需要更多空间:

```
evict least recently used leaves first
do not evict nodes used by running batch
use refcount to decide whether a node is evictable
```

为什么先 evict leaf?

因为 leaf 的祖先可能仍然是很多请求共享的 prefix。先删叶子可以保留更通用的公共前缀。

为什么 refcount 重要?

continuous batching 中, 有些请求正在用某个 cached prefix。如果 runtime 在中途 evict 它, 就会破坏正在运行的 decode。refcount 让 running batch 持有路径上的节点, 等 release 后才允许 evict。

8. Cache-aware scheduling

论文定义 cache hit rate:

```
cache_hit_rate =
    cached prompt tokens / total prompt tokens
```

如果 waiting queue 里有许多请求, 执行顺序会影响命中率。随机在不同主题之间切换, 会让 cache thrashing 变严重。

SGLang 的策略是:

```
match every waiting request against the radix tree
sort by matched prefix length
prefer requests with longer matched prefixes
```

Theorem 3.1 的直觉:

离线 batch 中, 如果 cache size 至少能容纳最长请求, 那么按 radix tree 的 DFS order 访问, 可以让每条边的 KV cache 只计算一次, 达到最优 hit rate。longest-shared-prefix-first 等价于一种 DFS order。

这不是说线上服务永远最优。在线请求不断到来, DFS 会被打断; greedy cache-aware scheduling 也可能造成 starvation。论文把和公平调度结合列为未来方向。

9. Frontend hint 为什么重要

论文很强调 frontend-runtime co-design。

在 fork 时, 前端知道多个分支共享当前 prompt。它可以先把 shared prefix 作为 hint 发送给 runtime, 确保 prefix 先进入 radix tree, 再发送各个分支 suffix。

如果 runtime 只看到各个完整 prompt, 它也可以做匹配, 但更难抓住程序内部的 fork 结构, 调度也更被动。Figure 8 的 ablation 说明, 禁用 frontend hint 和 frontend parallelism 都会降低性能。

所以 SGLang 的贡献不是单独的 radix tree, 而是:

```
language primitives expose structure
runtime uses that structure for cache and scheduling
```

10. Compressed FSM 和 jump-forward decoding

结构化输出常常要满足 JSON schema 或 regex。普通 constrained decoding 做法:

```
regex -> FSM
at each token:
    use current FSM state to mask illegal tokens
    decode exactly one token
```

问题是很多时候下一段是确定的。例如 JSON 开头:

```
{"summary": "
```

在这些 literal token 上, 合法下一步只有一个。普通 FSM 仍然 token-by-token decode, 浪费 forward pass。

SGLang 的 compressed FSM 做法:

```
find adjacent singular-transition edges
merge them into one compressed edge
when the path is forced, decode multiple tokens in one forward pass
```

Figure 4 对比了 normal FSM 和 compressed FSM。Appendix B 进一步说明, 先在字符或字符串级别建 FSM, 再压缩 singular transitions, 并处理 tokenization 细节。

本仓库对应文件:

- grammar_fsm.py: 简化 regex/FSM。
- jump_forward.py: 如果当前状态只有一个合法字符, 就一直往前跳。
- constrained_sampler.py: 根据 FSM 状态构造合法 token mask。
- sglang_original_minimal.py: 用 literal {"summary": " 演示 normal 需要 12 步, jump-forward 可以视为 1 个强制片段。

论文实验里, compressed FSM 在 JSON decoding benchmark 上把 throughput 提高 1.6x。另一个很关键的工程点是 FSM 预处理要复用; 如果每个请求都重新预处理, throughput 会低 2.4x。

11. API speculative execution

前几节主要面向 open-weight model, 因为 runtime 可以改模型执行流程。Section 5 讨论黑盒 API, 例如 GPT-4 或 GPT-3.5。

典型多调用模式:

```
context + "name:" + gen("name", stop="\n")
        + "job:" + gen("job", stop="\n")
```

朴素做法是两个 API call。第二次还要再次付 context 的输入 token 成本。

SGLang 的 API speculative execution:

first API call ignores stop and continues generating extra tokens
interpreter stores the extra output
later primitive tries to match and reuse it
if it matches the template, skip a later API call

这和 speculative decoding 有同一个味道: 先让模型多做一点可能有用的事, 再由程序结构验证是否可复用。

论文在 GPT-3.5 的 Wikipedia field extraction prompt 上测试, 少付约三倍 input token 成本。这里要读得保守: 这种技术依赖模板和 prompt engineering, 和 RadixAttention 那种 exact prefix reuse 不完全一样。

12. 实验证据链

12.1 Setup

模型:

- Llama-2 7B 和 70B。
- Mixtral-8x7B。
- LLaVA-v1.5-7B image。
- LLaVA-NeXT-34B video。
- OpenAI GPT-3.5 API。

硬件:

- 多数 open-weight 实验在 AWS EC2 G5 的 NVIDIA A10G 24GB 上。
- 7B 单卡。
- 大模型用 tensor parallelism。
- 另有 A100 80GB 实验。

Baselines:

- Guidance v0.1.8 with llama.cpp。
- vLLM v0.2.5 OpenAI-like API server。
- LMQL v0.7.3 with Hugging Face Transformers。

Workloads:

- MMLU 5-shot。
- HellaSwag 20-shot。
- ReAct agents。
- generative agents。
- tree-of-thought。
- skeleton-of-thought。
- branch-solve-merge LLM judge。
- JSON decoding。
- multi-turn chat short and long output。
- DSPy RAG pipeline。

Metrics:

- throughput: program instances per second。
- latency: single program average latency。

12.2 End-to-end results

论文主结果:

- SGLang throughput 最高提升 6.4x。
- latency 最高降低 3.7x。

不同 workload 的原因不同:

- MMLU: 复用 5-shot examples 的 KV cache。
- HellaSwag: 复用 examples 和 common question prefix, 属于 two-level sharing。
- ReAct/generative agents: 复用 agent template 和 previous calls。
- tree-of-thought/skeleton-of-thought: fork 分支并行, 并尽量复用 prefix。
- JSON decoding: compressed FSM 一次跳过多个确定 token。
- multi-turn chat: 复用聊天历史; 短输出收益更明显, 长输出 decode time 占主导, 收益较小。
- DSPy RAG: 复用 common context example。

论文报告这些 benchmark 的 cache hit rate 在 50% 到 99% 之间。Figure 13 显示 cache-aware scheduling 平均达到 optimal hit rate 的 96%。

12.3 Larger and multi-modal models

Mixtral-8x7B 和 Llama-70B 上的趋势类似, 说明优化不只在 7B 上有效。

Multi-modal Table 2:

- LLaVA-v1.5-7B image: 作者原始实现约 0.18 image/s, SGLang 约 1.15 image/s。
- LLaVA-NeXT-34B video: 作者原始实现约 0.02 frame/s, SGLang 约 0.10 frame/s。

为什么 multi-modal 也能受益?

因为同一图像或视频的 token 也可以作为 prefix 的一部分复用。SGLang 对图像输入做 hash, 把它作为 radix tree key 的一部分。

12.4 Production deployment

论文说 SGLang 已部署到 Chatbot Arena 服务 open-weight models。一个月观测:

- LLaVA-Next-34B 的 RadixAttention cache hit rate 为 52.4%。
- Vicuna-33B 的 hit rate 为 74.1%。
- cache hit 来自 common system messages、复用图像和 multi-turn chat history。
- Vicuna-33B first-token latency 平均降低 1.7x。

这段很重要, 因为它说明 cache reuse 不是只在合成 benchmark 中出现, 真实服务中也会出现。

12.5 Ablations

Figure 8 有三类信息。

Cache hit rate vs performance:

- cache hit rate 越高, batch size 越大。
- throughput 越高。
- first-token latency 和 total latency 越低。

RadixAttention components:

- No Cache: 不用 cache。
- No Tree Structure: 用简单表 cache, 不用 tree。
- FCFS Schedule: 不用 cache-aware scheduling。
- Random Schedule: 随机调度。
- No Frontend Parallelism: 禁用前端并行。
- No Frontend Hint: 禁用 fork hint。
- Full Optimization: 全开。

结论是这些组件都重要, 特别是它支持论文核心主张: 前端语言和后端 runtime 要协同。

Overhead:

ShareGPT 上没有 KV cache reuse 机会。100 个请求耗时 74.3 秒, 而维护 RadixAttention 数据结构只用 0.2 秒, 少于 0.3% overhead。因此作者认为它可以默认开启。

Compressed FSM:

JSON decoding 上 throughput 提高 1.6x; 如果每个请求都重新做 FSM preprocessing, throughput 会低 2.4x。

13. 和本仓库代码的连接

核心 radix tree:

`learning/sglang-radixattention/src/radix_tree.py`

对应论文机制:

- `Node.token_ids`: radix-compressed edge segment。
- `Node.kv_slots`: toy KV slot。
- `match`: prefix matching。
- `_split`: partial prefix sharing 时拆边。
- `insert`: 插入新 prompt, 统计 matched tokens。
- `acquire/release`: running request refcount。
- `evict`: LRU leaf-first eviction。

前端语言:

`learning/sglang-radixattention/src/frontend_lang.py`

对应:

- `Stream`: prompt state。
- `Gen`: generation primitive。
- `Select`: choice primitive。
- `fork`: 分支并行。
- `function`: 类似 `@sgl.function` 的装饰器。

结构化输出:

`learning/sglang-radixattention/src/grammar_fsm.py`

`learning/sglang-radixattention/src/jump_forward.py`

`learning/sglang-radixattention/src/constrained_sampler.py`

Capstone:

`learning/sglang-radixattention/src/agent_server.py`

它模拟 32 个 ReAct agent 共享长 system prompt, 并用 radix tree 统计 hit rate。

我补的机制实验:

```
learning/sclang-radixattention/src/sclang_original_minimal.py
learning/sclang-radixattention/src/tests/test_sclang_original_minimal.py
```

它输出:

- naive prefill tokens。
- RadixAttention 后需要计算的 prefill tokens。
- saved prefill tokens。
- hit rate。
- compressed FSM jump-forward 的 normal steps 与 jump steps。

运行:

```
.\\.\venv\\Scripts\\python.exe `
learning\\sclang-radixattention\\src\\sclang_original_minimal.py

.\\.\venv\\Scripts\\python.exe -m pytest `
learning\\sclang-radixattention\\src\\tests -q
```

这不是生产 runtime, 但它能让你把论文 claim 落到本机可测指标:

paper:

cache shared prefixes in a radix tree

toy:

naive_prefill_tokens - radix_prefill_tokens = saved_prefill_tokens

14. 这篇论文没有证明什么

SGLang 很实用, 但它不是所有 serving 场景的万能解。

它没有证明:

- 所有 workload 都有大量 prefix sharing。
- 长输出场景一定有明显收益。
- cache-aware scheduling 不会带来公平性问题。
- exact prefix matching 能覆盖语义相似但 token 不同的 prompt。
- compressed FSM 对所有 schema 都同样高效。
- API speculative execution 永远准确或不会改变应用逻辑。
- compiler mode 可以处理所有 data-dependent control flow。
- prompt code movement 一定保持模型行为不变。

论文自己的 future directions 也指出:

- RadixAttention 扩展到 DRAM/Disk 等多层 memory hierarchy。
- fuzzy semantic matching。
- 更高层 primitives。
- 修复 cache-aware scheduling starvation。
- 更强 compiler, 做 scheduling 和 memory planning。

读这篇时要保持一个系统工程判断: 它的收益来自 workload structure。如果 workload 没有共享前缀、没有分支、没有结构化输出, SGLang 的优势会小很多。

15. 和后续工作的关系

SGLang 和 vLLM/PagedAttention 是互补关系。

vLLM 解决:

KV cache memory fragmentation
continuous batching
paged attention blocks

SGLang 解决:

LM program structure
multi-level prefix sharing
frontend-runtime co-design
structured output fast path

后续你会看到的相关方向:

- xgrammar 和 Outlines: 更强的 grammar/constrained decoding。
- HydraGen 和 PromptCache: shared prefix 或 modular KV reuse。
- DistServe: prefill/decode disaggregation。
- SGLang serving/runtime 后续版本: 更强 batching、spec decode、grammar、multi-modal support。
- Agent runtime: 多 agent、tool calling、parallel branches 的系统化调度。

16. 用 AI agent 学这篇论文的正确方式

不要只让 agent 问 “什么是 RadixAttention”。你要让它把程序结构和 runtime 机制绑在一起考你。

可以这样提问:

我正在学习 SGLang 论文。

请一次只问我一个问题。

问题必须来自下面八类之一:

1. LM program 为什么不是普通 single completion
2. gen/select/fork/join 暴露了什么结构
3. KV cache 为什么只依赖 prefix tokens
4. radix tree 的 match/split/insert/evict 如何对应 Figure 3
5. cache-aware scheduling 和 Theorem 3.1 的直觉
6. compressed FSM 为什么能 jump forward
7. Figure 5/6/8/13 的证据链
8. 本仓库 radix_tree.py 或 sglang_original_minimal.py 的代码映射

我回答后, 请指出漏洞。

每次都要求我把答案映射到论文 section、图表或本地函数。

最后让我用 200 字闭卷复述。

一个很好的自测问题:

给定 32 个 ReAct agent 都共享同一个 system prompt,
为什么 naive serving 会重复算 prefix,
而 RadixAttention 可以把它变成 cache hit?
请同时解释 refcount 和 LRU leaf eviction。

17. 读完后的闭卷复述模板

按这个模板复述:

SGLang 的问题背景是:

...

LM program 和普通 completion 的区别是:

...

前端语言提供:

...

RadixAttention 的数据结构是:

...

Figure 3 里的 split 和 eviction 说明:

...

cache-aware scheduling 的直觉是:

...

compressed FSM 的作用是:

...

实验最关键的证据是:

...

限制是:

...

我能在本仓库看到的最小实现是:

...

如果你能不看 guide 填完这段, 并且能打开 radix_tree.py 指出 match、_split、insert、evict 分别对应论文 Figure 3 哪些动作, 这篇 SGLang 就真正进脑子了。